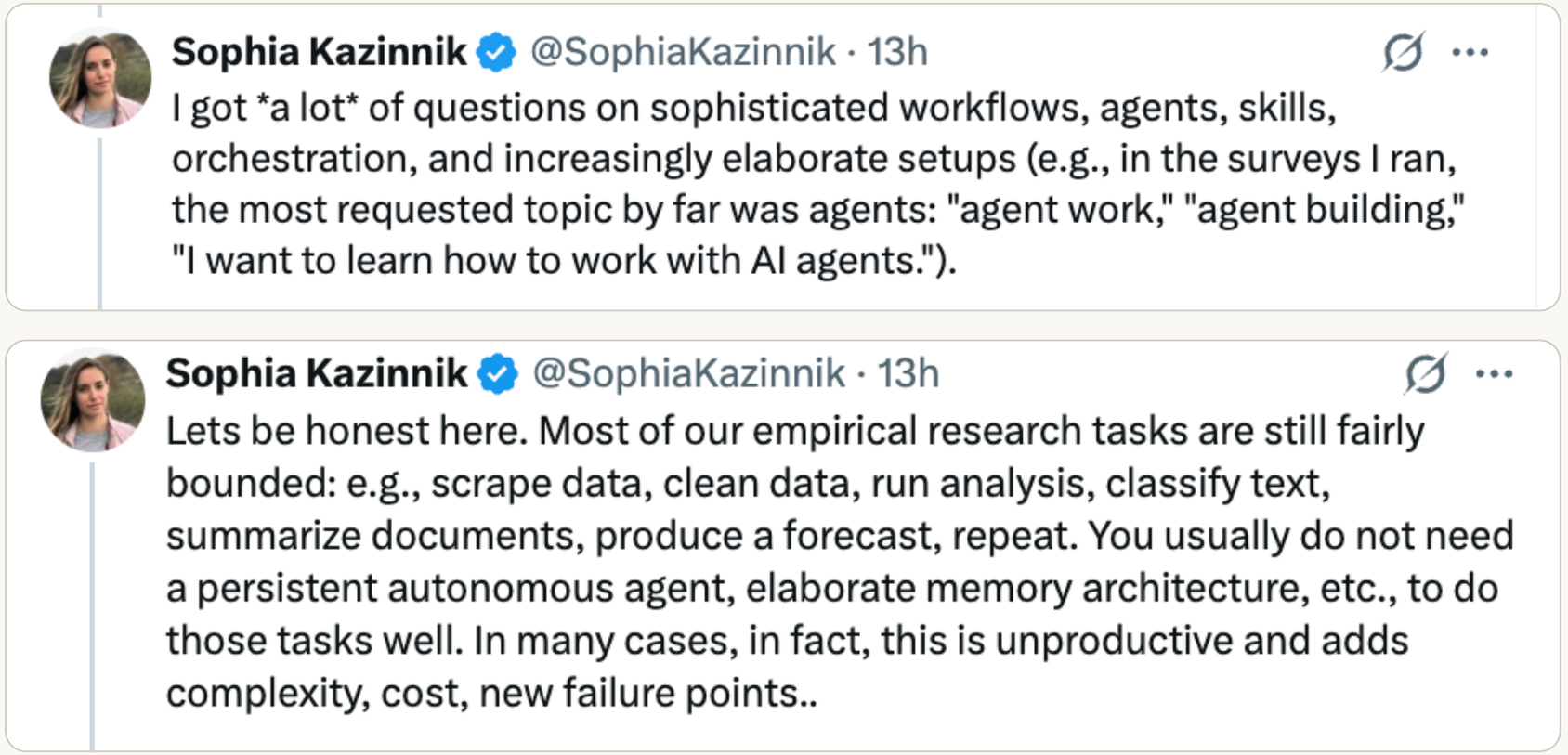
Practical AI for Academics

Session 2 - Practical workflows

Claes Bäckman
Leibniz Institute for Financial Research SAFE

Most research writing tasks are bounded and do not need skills



Sophia Kazinnik @SophiaKazinnik · 13h

I got *a lot* of questions on sophisticated workflows, agents, skills, orchestration, and increasingly elaborate setups (e.g., in the surveys I ran, the most requested topic by far was agents: "agent work," "agent building," "I want to learn how to work with AI agents.").

Sophia Kazinnik @SophiaKazinnik · 13h

Lets be honest here. Most of our empirical research tasks are still fairly bounded: e.g., scrape data, clean data, run analysis, classify text, summarize documents, produce a forecast, repeat. You usually do not need a persistent autonomous agent, elaborate memory architecture, etc., to do those tasks well. In many cases, in fact, this is unproductive and adds complexity, cost, new failure points..

Sophia Kazinnik on X (x.com/SophiaKazinnik). Screenshots as posted.

Before anything: the folder, the panel, the skills

1. VS Code is open with the Claude Code panel in the sidebar. Five steps if not.
2. **Your project folder is open, not a single file.** File, Open Folder. The folder holds the draft you brought and the PDFs of your own writing.
3. **The skills are installed.** Type `/` in the panel: `/review-paper` should appear in the list. If it does not, installing the skills takes two minutes; do it now with a neighbour.

`/name` runs a skill, a saved workflow; `@file` points the agent at a file in the folder.

Claude Code needs a paid plan (Pro, Max, Team or Enterprise). On a free account, pair up with a neighbour for the exercises.

Context improves AI tools tremendously, and the goal is to provide the right context.

The agenda for the session

1
Referee report
`> /review-paper`
then verify

2
Slides
`> /explorable-deck`
not a Beamer parrot

3
Writing
`> /voice-extractor`
your voice as a file

4
Paper and code
`> /review-paper-cod…`
does the code do what the paper says?

5
Teaching
`> from @notes/week3…`
write 15 questions

6
Secure data
`> read @codebook.md`
what never leaves the server

Every block is the same move as this morning's argument: put the right files in the folder, type one command, and read what comes back as a draft to check, not an answer to accept. Blocks 1, 3 and 5 have a timed exercise; block 4 has an optional one. **Break after block 3, around 15:30.**

Slides, skills and guides for both sessions: claesbackman.com/lugano.

1 · A referee report on your paper

```
> /review-paper
```

A pre-submission referee report on your own paper

Exercise 1 · a referee report on your own draft

10 min

1. In the Claude Code panel, type `/review-paper` and the journal you are aiming at, on one line. The draft must be LaTeX or markdown.
2. Click *Allow* when it asks to read files or run commands. Then let it run while we talk.
3. Short on time? Other options are:
   `/review-paper-light` takes about a minute.
   `/review-paper-checks` looks at spelling and grammar, consistency, and unsupported claims.
   No skill at all: type *Read the paper in this folder and review it as a referee for [journal]: identification, unsupported claims, tables.*

Debrief at the end of this block: *what did it catch that you would not have seen? What did it get wrong? What did the adversary you launch later find that the report missed?*

A skill is a markdown file the agent loads when you type its name

```
---
name: review-paper
description: Run a pre-submission referee
  report for an academic paper targeting
  a specified journal
---
You are coordinating a rigorous pre-submission
review of an academic economics paper. You will
run specialized review agents in parallel and
consolidate their findings into a structured
report.
```

It lives at `.claude/skills/review-paper/SKILL.md` in the project, or under `~/.claude/skills/` for every project. Typing `/review-paper` makes the agent follows the file.

663

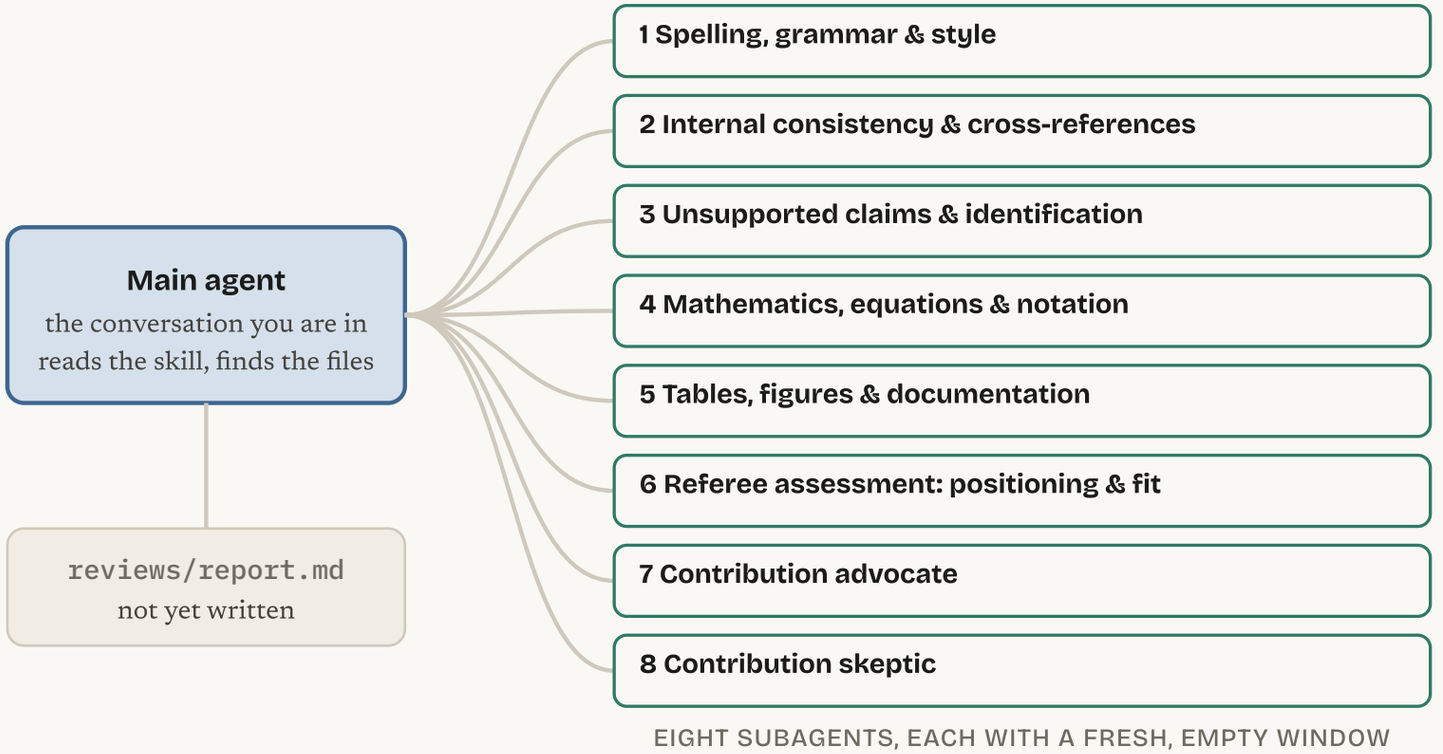
lines of markdown: about ten pages of instructions you can read, fork and edit

8

reviewers it launches, each with one job

Very easy to make your own skill if you keep repeating prompts: say *make this a skill* and the agent writes the file.

Eight narrow reviewers beat one diffuse reader



Run /review-paper Reset Press run: the main agent sends the paper to all eight at once.

The eight reviewers are the ones the installed skill launches; the timing is stylised. Teal: a reviewer with one job. The strip along each row is its window: a grey stub, the whole paper in brick, the rest empty. Blue: the main agent, which finds the files and consolidates.

But the agent that wrote the report will defend it

Ask the agent that wrote the report to check it



system your paper its own report the conversation so far

Its own answer is in the window. Everything it reads next is read in the light of having written it; it has momentum and a position to defend.

Spawn a fresh subagent to attack it



system your paper the report, as a file “read this as a hostile referee”

Same paper, same report, no history. The report is a file it was handed, not a position it took, and it reads both cold.

Stylised: the widths are not measured. The same report is blue in the first window, where it is the agent’s own output, and brick in the second, where it is a file handed to a reviewer. Both reviewers see the paper.

You still need to verify the report

Exercise 1b, then debrief

2 + 5 min

1. When your report is in, type in the same conversation: *Launch a subagent to critique this report as a hostile referee. Report, do not fix.*
2. Read the report against your two guesses. What did it catch that you would not have seen? What did it get wrong?
3. What did the adversary find that the report missed, or overstated?
4. Which of the eight sections would you read first next time, and why?

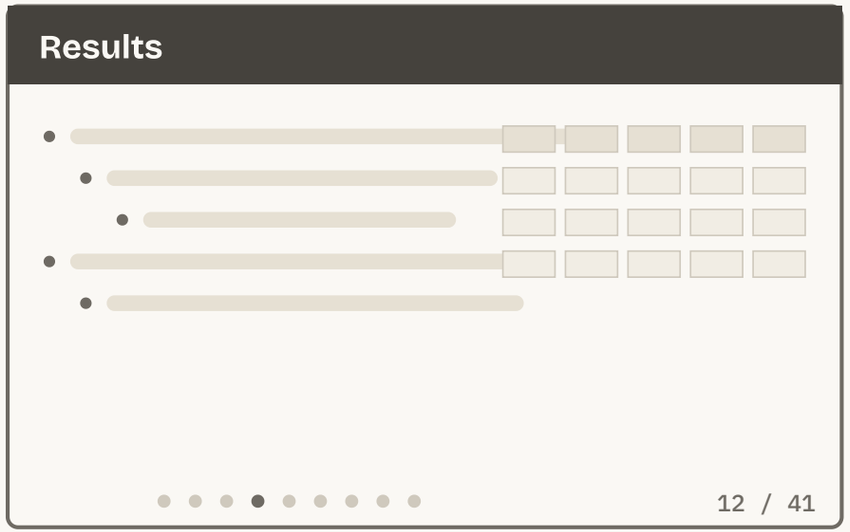
Feedback becomes cheap enough that you ask for it before submitting, not after the desk reject.

Other tools generate referee reports too: a short list.

2 · Slides and communication

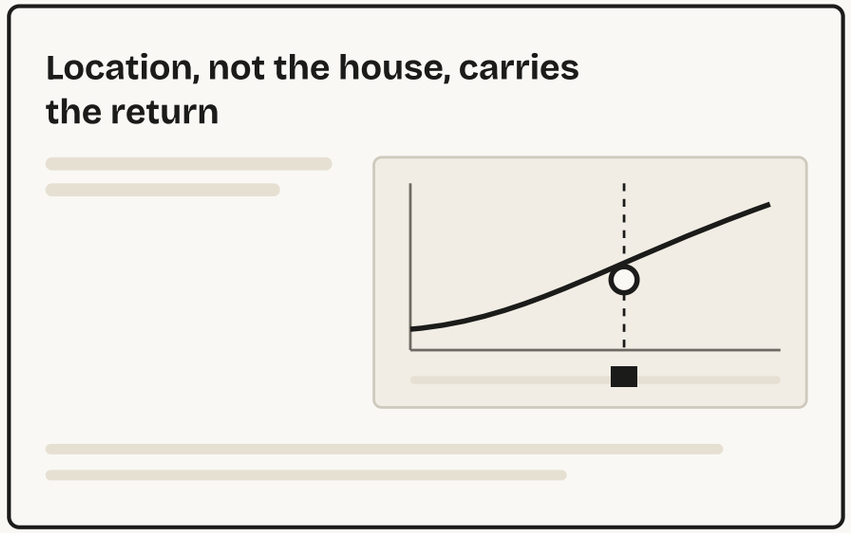
```
> /explorable-deck @paper.tex
```

Ask for academic slides and you get a stochastic Beamer parrot



the parrot

a working deck in twenty minutes
that looks like every other deck online



an explorable deck

a claim for a title, one idea, a live stage
the new thing, not the old thing faster

Stylised: two slides drawn from memory, not screenshots. Left, what “make a Beamer deck from the paper” produces. Right, one slide from an explorable deck of a housing-returns paper, with its handle. Both take about twenty minutes to draft.

AI should let us do new things, not only old things faster

- Nicky Case's explorable explanations, Bret Victor, Steven Strogatz: a clear narrative, interactive elements that carry the message, a clean visual language.

  This deck, and this morning's, were built with it from a Beamer source, and then edited.

  Examples: a housing-returns paper and a paper on financial advice from LLMs. The skill: github.com/claesbackman/AI-research-feedback, `Skills/explorable-deck`. Adapted from Nityesh Agarwal's skill for explorable websites.

But it comes with some caveats

The paper must be good

The deck is built from the paper. A muddled paper gives a fluent deck about the wrong thing.

You need to practise

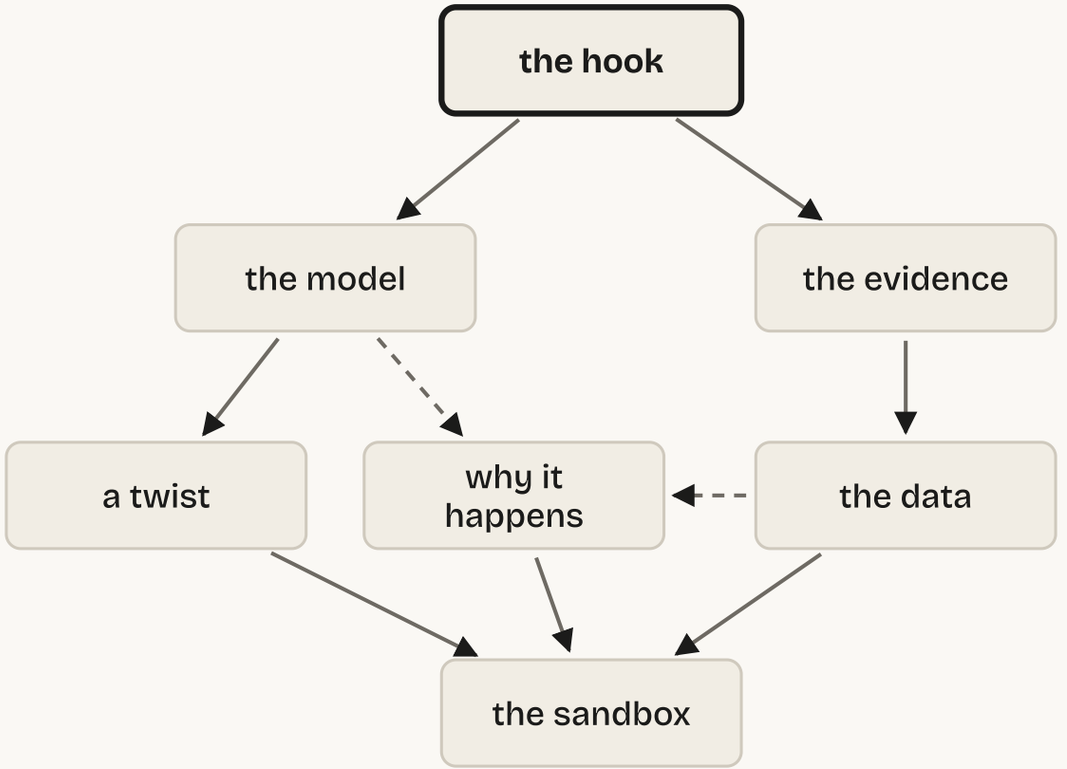
Practise more than before: making the slides by hand was how you learned your own talk, and that part is gone.

Other fields have worked out how to explain their work

`/explorable-explanation` builds a website as a non-linear tree of short HTML decks, one idea per slide, with forks placed where the reader becomes curious.

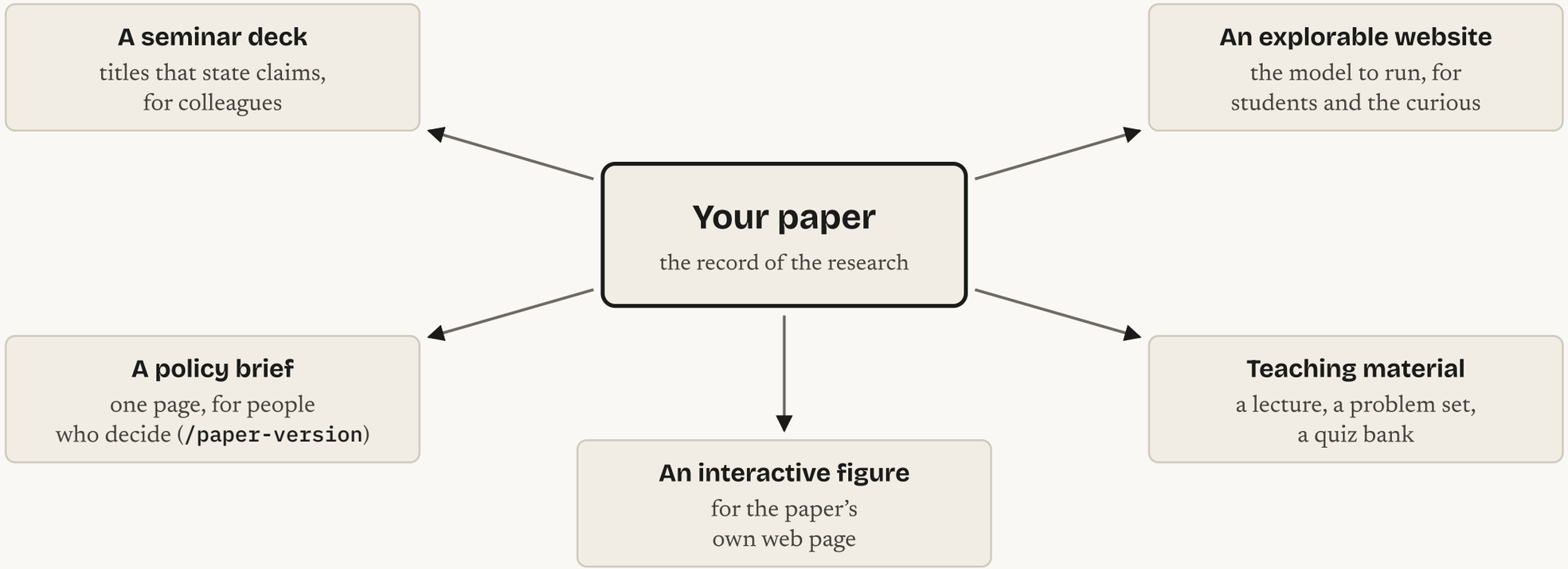
How it works

1. Reads the source material, then writes `plan.md` like a script: the hook, what the reader sees and does, and where the branches go
2. Shows you the first batch before it builds the rest
3. Screenshots every slide and reworks the ones that break the craft rules
4. Sends adversarial subagents to judge writing, interface, visuals and teaching
5. A “creative director” subagent gives it a visual identity; an impatient-reader subagent flags every slide where the flow breaks



Each box is a short deck. Dashed arrows are forks the reader chooses from inside a slide.

We do not have to be bound by the PDF



Innovations are already happening

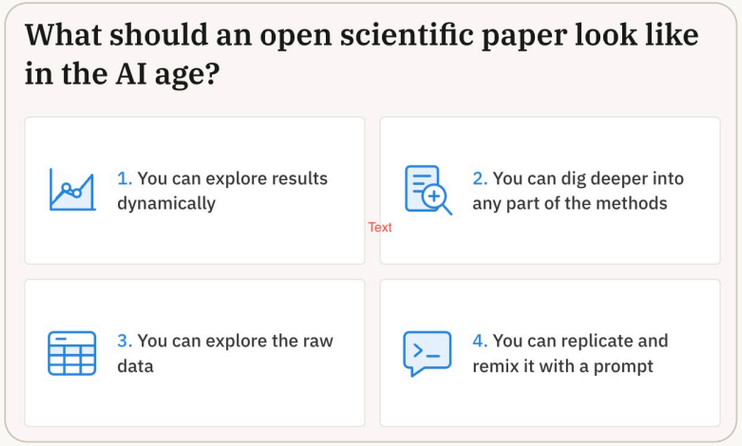
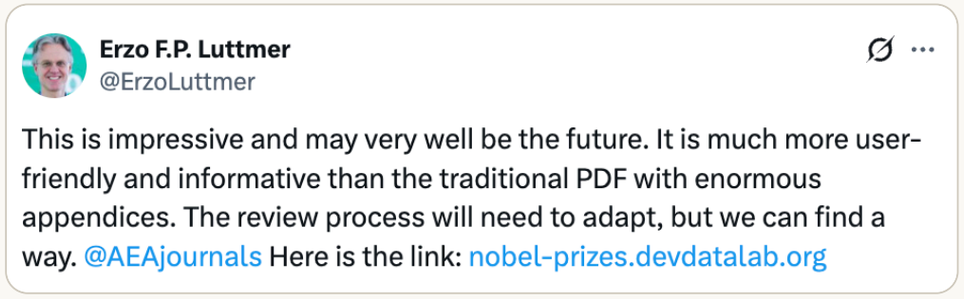


Figure from https://nobel-prizes.devdatalab.org/



3 · Writing with agentic AI

```
> /voice-extractor sample-papers
```

Use AI to edit your draft; do not edit the AI's draft

Writing is thinking

Remove too much of the writing friction and it gets harder to find out what your paper is about. The first draft is where the argument is discovered.

Readers discount prose they think is generated

Once a reader decides a paragraph was generated, by instinct or with a detector, they discount the idea in it. That is a cost to the idea, not to the prose.

Most of the value is not in having AI write your text. Use it for what it is good at, in your voice, and learn to ignore some of what it says.

A voice file puts your prose in the window

> *When editing my prose, match the voice in* `voice.md`*. Do not use em-dashes, bullet lists, or passive constructions unless the original does. Do not use "it's not x, it's y" constructions.*

What goes in. Five to ten thousand words of your own prose: introductions, abstracts, paragraphs you are proud of. The skill picks them out of the PDFs you give it.

What stays out. Coauthored passages that are not yours. Anything AI-edited.

Where it goes. Next to `CLAUDE.md`. Point at it from there, so every session loads your voice, or with `@voice.md` only when editing, to keep the permanent window small. The files that hold durable context.

Analyze your own voice

Exercise 2 · `/voice-extractor` **15 min**

1. Inside the project folder, make a folder `sample-papers` with three or four PDFs of your own writing. Coauthored? Pick the papers where the introduction is yours.
2. Type `/voice-extractor sample-papers`. No skill? Type *Read the PDFs in sample-papers and write voice.md: how I build paragraphs, sentence length, words I use and avoid, with quoted examples.*
3. Read `voice.md` and delete or edit anything that does not sound like you.
4. Add one line to `CLAUDE.md`: "When editing my prose, match the voice in voice.md." No `CLAUDE.md` yet? Type `/init` first.

Debrief: *which sentence in* `voice.md` *did you delete first? Did it name a habit you did not know you had?*

/prose-revision fixes the order of ideas before the choice of words

What comes back

A diagnosis of the idea flow first. Then two or three rewrites that differ in how far they restructure, and a short note on the trade-off between them. Claims, citations and notation come through unchanged.

A skill in the library, after a method described by alz_zyd on X (x.com/alz_zyd_).

Where it stops

It will not add or drop an idea without saying so, and it is not a proofreader. Revision first, the typo pass last.

But: AI detectors can tell AI generated writing

Pangram is the most-cited detector at the moment: a classifier trained on the fingerprint of model text, with a low false-positive rate in its own reporting.

The current state of play on writing

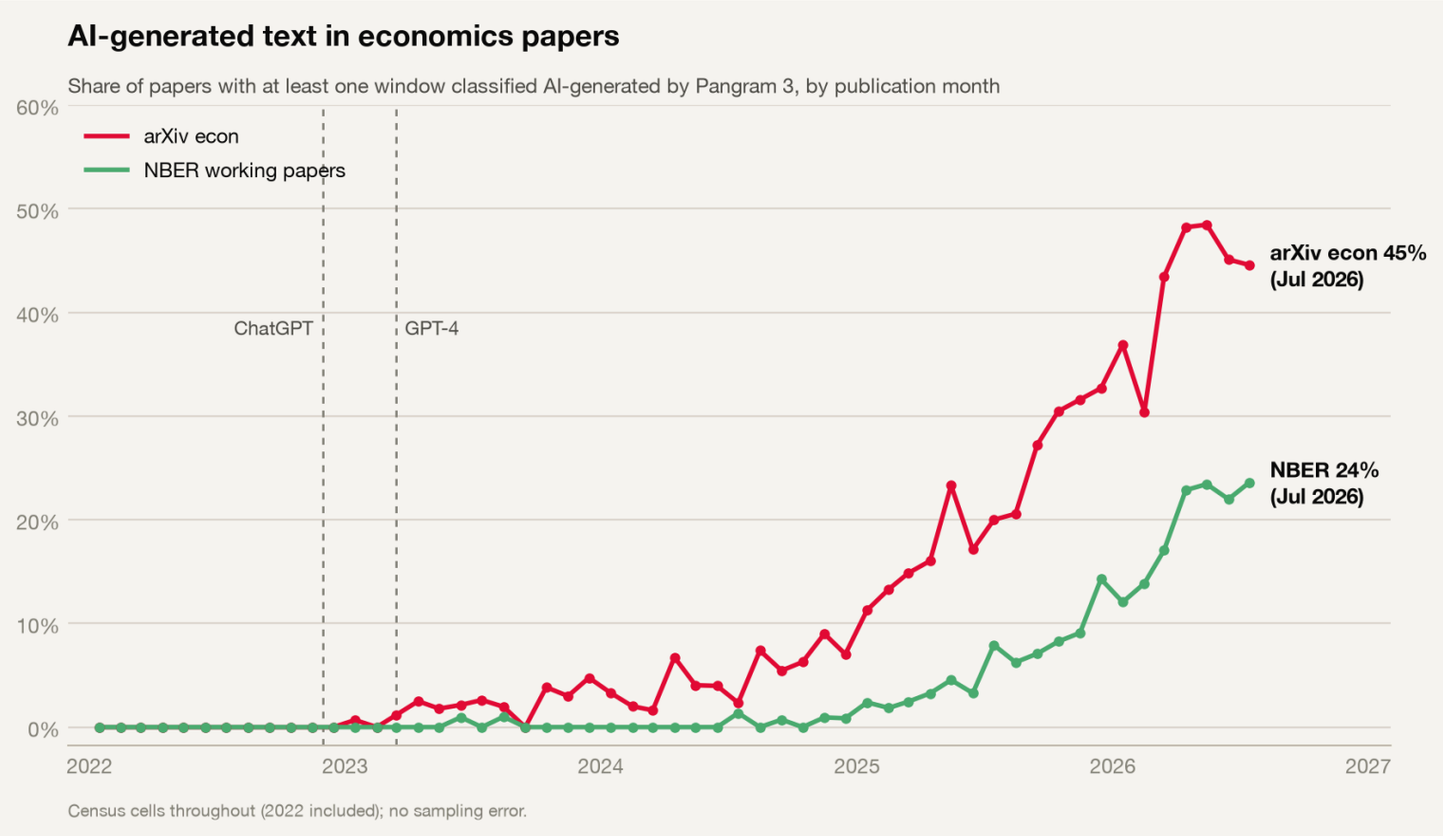
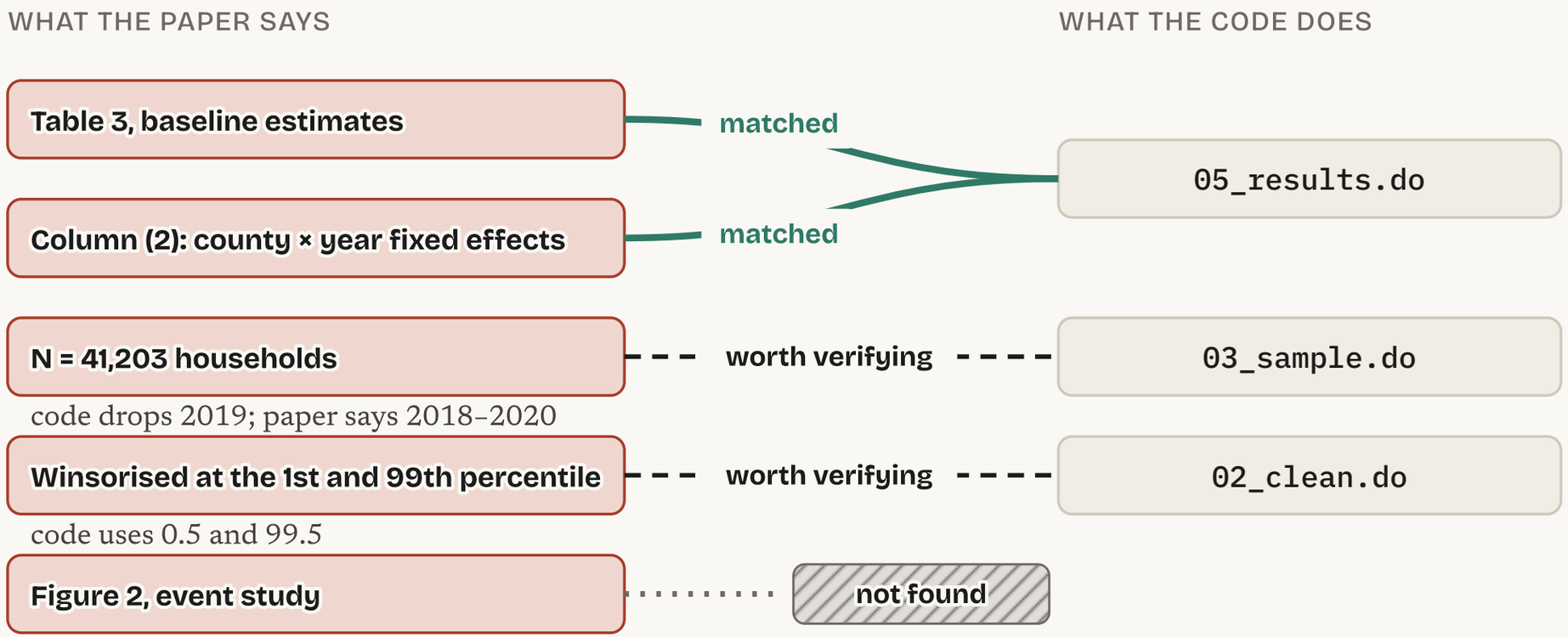


Figure from Paul Goldsmith-Pinkham, A Causal Affair on Substack

4 · Verifying code and changes

```
> /review-paper-code
```

/review-paper-code maps the paper's claims to the scripts that produce them



A stylised example of the paper–code map the skill writes. The five rows are invented for the slide; the three statuses are the report's. Brick: claims read from the paper. Teal: the code does what the paper says. Dashed: the two disagree, and that row is read first. Hatched: no script produces it.

Git diff is a very useful command for agents

A **repository** is a folder with a history. A **commit** is a saved snapshot of it. A **diff** is the list of lines that changed between two snapshots.

An agent that sees only the diff knows exactly what is new.

`/explain-diff`: the check for this block

Point it at a repository and it writes one offline HTML page about the last change: what the code did before, what changed and why, what it does to the results, and a five-question quiz.

GitHub or Dropbox?

	GITHUB	DROPBOX
Sync	Manual push and pull, in GitHub Desktop, no terminal	Automatic, within seconds
History	Full history, diffs, branches	Dropbox revisions only
Best for	Versioned drafts; the review skills	Live coauthoring in Overleaf
Conflicts	A merge you resolve	“Conflicted copy” files

5 · Teaching

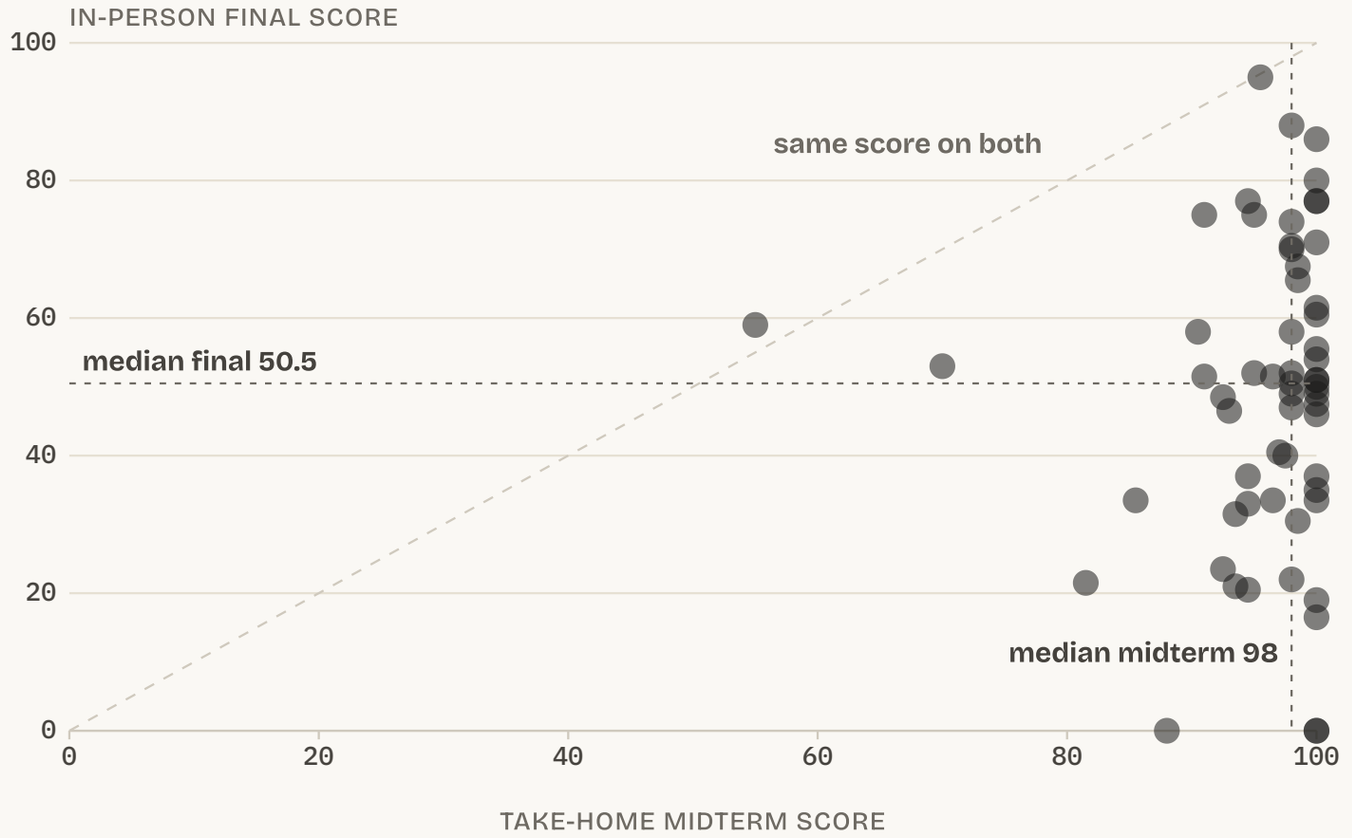
```
> from @notes/week3.md, write 15 multiple-choice questions at three levels
```

Take-home grades stopped predicting in-person grades

The median take-home mark was 98; the median in-person final, for the same students, was 50.5.

Three things this changes. A good take-home grade no longer shows the student can do the thing. Students will need to use AI well later, so banning it everywhere leaves them untrained. And tutoring, feedback and individual practice used to be too expensive; AI should let us teach more.

Bryan (2026). The eight rules and three things to build are in the appendix.



Measured. One dot per student, 59 students who took both exams in Serrano's ECON 1170 after the final was moved in-person; 18 dropped the course and 9 did not sit the final. Values read off the chart reproduced in Bryan (2026). The same chart as this morning.

Build a weekly quiz bank from your notes

Exercise 3 · a quiz bank

10 min

1. Save one week's lecture notes as `notes/week3.md` in the project folder, as markdown or LaTeX; a PDF or `.pptx` converts first. No notes? Use the example notes on the resources page.
2. Type the prompt:

> *From* `notes/week3.md`*, write 15 multiple-choice questions at three levels, with distractors built from common misconceptions and a one-sentence explanation each.*

3. Read every question. **Verify every answer key yourself.**

6 · Working with secure data

```
> read @codebook.md and the first rows of @sim_data.csv, then write analysis.do
```

Your draft can go into the window; register data and server output cannot

What went in today, and may

Your draft, the PDFs of your own writing, a codebook, a simulated dataset. Files the agent reads go to the provider's servers under your subscription's terms.

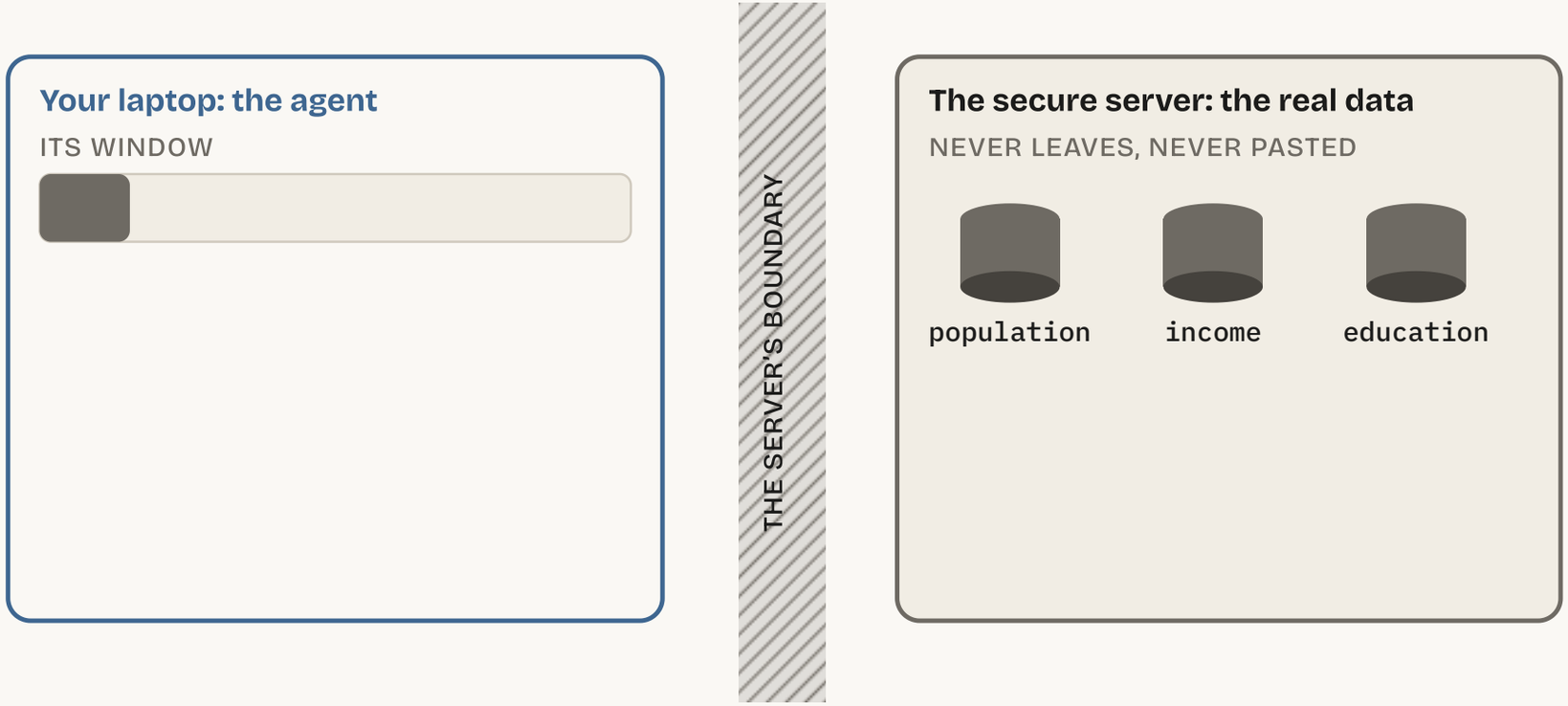
Check the data setting once. Some consumer tiers may use chats to train by default; team and enterprise tiers do not. Know which you are on before anything sensitive.

What never goes in

Microdata, register extracts, any number from a log or table produced on the secure server, anything under a data-use agreement. Not into the agent, not into a chat window.

When in doubt, abstract it: a made-up example, a synthetic dataset. And check the institutional policy once, before the first paste.

The agent never sees the data, but it sees everything that explains the data



Step Reset Press **step**. Six steps; the wall is never crossed by data.

Stylised. Brick: what is in the agent's window. Grey: the registers the agent never sees. The dashed arrow is you carrying the do-file. The struck-out bubble: pasting the log into a chat window is the agent reading it. What crosses back is what you type: an error code or a request, never a result.

Two files to keep local: a codebook in markdown and a simulated dataset

1 · Codebook in markdown

Convert the PDF once with `/pdf-to-markdown`. Variable names, labels, value ranges, missing-value codes.

The agent reads it before it writes a `recode`, a `label define` or a `merge`.

2 · Simulated dataset

Same variable names, types and ranges; not the real values. Some replication packages ship one.

Or ask the agent to generate it from the codebook: one prompt, fifty thousand rows.

But synthetic data tests that the code runs, not that it is right

Tests that the code runs

Variable names, merge syntax, reshapes, type handling, the shape of the panel. Everything that would fail on the schema fails on your laptop, before you enter the secure environment.

Does not test that it is right

Correlations, magnitudes, selection, missingness, sign conventions, duplicate keys and gaps that only the real values produce. The fake data contains what someone thought to put in it.

Your turn

The folder is yours. So is the check.

```
> 
```

Tonight: keep the folder, keep `voice.md`, keep the line in `CLAUDE.md`. Tomorrow, while the coffee brews: `/review-paper-light` on something small, and read it as a draft.

Questions now or later: claes.backman@gmail.com. Slides, skills, guides: claesbackman.com/lugano

Appendix

```
> the rooms, for questions
```

Five steps get VS Code and Claude Code working in under five minutes

1. Open VS Code and the Extensions panel (`Ctrl+Shift+X`, `Cmd+Shift+X` on a Mac).
2. Search "Claude Code" and install *Claude Code for VS Code* (extension ID `anthropic.claude-code`).
3. Restart VS Code. Claude Code opens in the sidebar.
4. Sign in with your Claude account (Pro, Max, Team and Enterprise all include Claude Code), or an API key.
5. Open your **full project folder** (File, Open Folder), never a single file.

What an IDE is. Editor, file explorer, terminal and extensions in one window; think of it as Word or RStudio for code, LaTeX, markdown and plain text. VS Code is the dominant free one.

Then: the extensions, every session, installing the skills. Back to the check-in.

Installing the skills is a copy into one hidden folder

1. Download the zip from claesbackman.com/lugano, or paste the one-line command from that page into the VS Code terminal (`Ctrl+``). Source and updates: github.com/claesbackman/AI-research-feedback. `/voice-extractor` is Mihail Velikov's and is linked from the same page.
2. Unzip, then copy each skill folder (`review-paper`, `review-paper-light`, `voice-extractor`, ...) into `~/.claude/skills/`, a hidden folder under your user folder. Windows: `C:\Users\you\.claude\skills`; to see hidden folders, View, Show, Hidden items in File Explorer.
3. A skill in `~/.claude/skills/` works in every project; one in `project/.claude/skills/` only in that project.
4. Start a new Claude Code conversation. Type `/` and the skills appear in the list.

Back to the check-in or the skill slide.

A handful of extensions cover Stata, Python, LaTeX and data

STACK	EXTENSION ID	WHAT IT DOES
Stata	`tmonk.stata-workbench`	Run Stata from the editor
Python	`ms-python.python`	Syntax, IntelliSense, run button
LaTeX	`james-yu.latex-workshop`	Compile on save, side-by-side PDF preview, inline errors
Data	`mechatroner.rainbow-csv`	Colour-coded CSV columns; SQL-like filtering
	`tomoki1207.pdf`	Renders PDFs inline: read codebooks next to your code

Install Python even if you do not write it. The agent uses Python as glue for many different tasks.

Back to setup.

Every session: open the folder, set context, then prompt

1. Open VS Code and the project folder (File, Open Folder).
2. Open the Claude Code panel in the sidebar.
3. Confirm Claude is pointed at the full project folder, not a single file.
4. Send a first orienting prompt.

> *Read the files in this folder and tell me what this project is about.*

Expected behaviour. Claude reads the folder, summarises the structure, and asks what you want to do next.

Then: permissions, a session is a conversation. Back to setup.

Permissions follow a tiered model, and auto mode is now the default

Plan mode	`reads only`	Reads and proposes; cannot write.
Manual	`asks first`	Asks before each edit or command.
Edit automatically	`edits files`	Edits your selected text or the whole file without asking.
Auto mode	`the default now`	Approves actions that pass a safety check and pauses for anything risky.

Claude Code is meant to stay inside the folder you opened; do not rely on it for a folder that holds anything sensitive.
`/permissions` shows the current rules; allow- or block-list specific commands (`Rscript`, `pdflatex`, `rm`) in `settings.json`.
Manual mode for the first week is a reasonable choice.

Back to every session.

A session is a conversation, not a query

1. **Prompt:** specific, with the context it needs.
2. **Response:** a draft, not an answer.
3. **You react:** accept, correct, redirect. The next turn builds on your corrections; ten to twenty turns is normal.

Continue the conversation. “Shorter, and use the Danish case.” “You misread Table 2; look again.” “Now the same for Section 3.”

Off track? Kill it and restart with a better first prompt

Signs: it repeats a mistake, argues in circles, forgets an earlier instruction, or three turns go on fixing one thing.

Start fresh and put what you learned into the first prompt, or into `CLAUDE.md` so you never repeat it. Signs of drift.

Back to every session.

Signs of session drift

1. Claude ignores `CLAUDE.md`
2. Repeats mistakes you already corrected
3. Invents files that do not exist
4. Gives circular answers
5. Forgets the last three turns

Fix. Start a fresh conversation (`/clear`), or close and reopen. Re-state the task. Point at the relevant file again.

Thirty turns, or the task has shifted: start fresh.

Back to a session is a conversation.

A few files hold your durable project context

FILE	USE IT FOR
`project/CLAUDE.md`	Project instructions: dataset structure, sample restrictions, paper title, coauthor conventions, which script produces which output
`~/.claude/CLAUDE.md`	Personal preferences across every project: writing style, default tone, citation conventions. Project instructions load later and usually carry more weight
`voice.md`	Writing samples: paragraphs, abstracts and tone notes Claude should match when editing prose
`.claude/skills/…`	Reusable workflows: named tasks you run again without rewriting the prompt

Keep secrets and raw confidential data **out of all of them**: they are read into context every session and sent to the model provider.

Then: bootstrap with /init, the starter template. Back to the voice file.

/init writes a first CLAUDE.md in two minutes

The built-in command. Type `/init` in the Claude Code panel. Or spell it out:

> *Read this project folder and draft a* `CLAUDE.md` *file with the project structure, conventions, and instructions you should follow. Do not edit anything else yet.*

What Claude does. Reads the folder, infers languages, structure and conventions, and writes a draft.

Then you edit it. Add tone preferences, things to avoid, and the sample restrictions it could not infer. The draft is the starting point, not the final word.

Back to the files or exercise 2.

CLAUDE.md starter template

```
# About me
I am a researcher in [field].

# How I want you to work with me
- Ask clarifying questions before long output.
- Critical, skeptical tone in feedback. Do not flatter.
- When editing, preserve my voice (see voice.md).
  No generic LLM phrasing.
- No bullet lists or passive voice in formal writing.
- Cite the line or section you are commenting on.

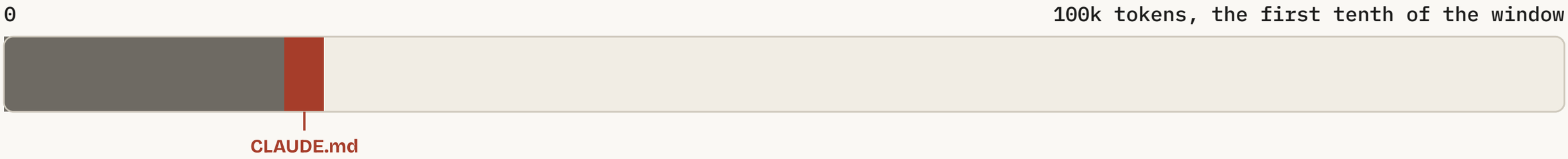
# Things to avoid
- Do not fabricate citations.
- Do not over-claim causality.
- No emoji or markdown decorations in formal docs.
```

Always loaded, every session: the things you would otherwise repeat in every prompt. Keep it to a page; long workflows go in skills.

Back to CLAUDE.md and skills or the files.

Keep CLAUDE.md short; put long workflows in skills

Two ways to put standing text in front of the agent. `CLAUDE.md` is read at the start of every session: who you are, your conventions, how you want it to work. A skill is read only when you type its name. Both are yours; only one is permanent. The starter CLAUDE.md; `/init` writes a first draft.



system prompt and tool definitions (the provider's) · CLAUDE.md, yours, every session · /// the skill, yours, only when called

type /review-paper Every session starts like this. CLAUDE.md is about 2,500 tokens, in every window, every time. Nothing of the skill is here yet.

Stylised token counts with real orders of magnitude: a million-token window, of which this strip is the first tenth. Brick is text you wrote for the agent; solid is loaded every session, hatched only when called. Grey is the provider's.

The skill library

SKILL	USE
`/review-paper`	Pre-submission referee report, 8 reviewers
`/review-paper-light`	Fast 2-reviewer version, about 1 minute
`/review-paper-code`	Paper and code alignment review
`/review-grant`	Pre-submission grant review, 6 reviewers
`/review-pap`	Pre-analysis-plan review
`/audit-analysis`	Adversarial audit of changed analysis code
`/explain-diff`	Explain a code change as an HTML page with a quiz
`/paper-version`	Paper to policy brief, 1-page or 5-page summary
`/pdf-to-markdown`	Convert PDFs to readable markdown
`/prose-revision`	Revise idea flow before wording
`/voice-extractor`	Write `voice.md` from PDFs of your own prose (Mihail Velikov)

Mine are at github.com/claesbackman/AI-research-feedback (MIT); all are linked from claesbackman.com/lugano. Fork them, modify them, make them yours. Back to the skill slide.

File formats: text is free, binaries cost a conversion

FORMAT	HANDLING
`.md`, `.tex`, `.txt`, `.bib`	Native, no conversion
`.py`, `.R`, `.do`, `.ipynb`	Native: read, edit, run
`.csv`	Native for a few thousand rows; larger files it inspects through code, a few rows at a time
`.docx`, `.pptx`	Convert: `pandoc file.docx -o file.md`, or ask the agent to
`.xlsx`	Export to CSV first
`.dta`, `.rds`	Read through code, not as text
PDF, born-digital and short	Native; expect table mangling
PDF, long, scanned or with equations	Convert to markdown first: `/pdf-to-markdown`

Back to exercise 1 or the two files.

You can link Overleaf to your own computer and work in that folder

Why bother. Overleaf has no Claude Code, no Codex, no Copilot, no proper diff view, no terminal. The fix: write in VS Code and let Overleaf follow what you produce.

Requirement. Both routes need Overleaf Premium; many universities provide it through the institutional login.

Two routes. GitHub via GitHub Desktop for a history, click to commit, no terminal. Dropbox for live coauthoring, no commit step. Click by click.

claesbackman.com/overleaf-vscode-guide; Overleaf documentation. Back to GitHub or Dropbox.

The two Overleaf routes, click by click

Setup A · GitHub via GitHub Desktop

- Overleaf: Menu, Integrations, GitHub, **Export Project to GitHub**
- GitHub Desktop: File, Clone repository; pick the repo and a local folder
- Open that folder in VS Code and edit
- GitHub Desktop: one-line summary, **Commit to main**, **Push origin**
- Overleaf: Menu, GitHub, **Pull GitHub changes into Overleaf**
- Coauthor edited in Overleaf? **Push Overleaf changes to GitHub**, then pull in GitHub Desktop

Accept the TeX `.gitignore` template: no `.pdf` or `.aux` in the history.

Setup B · Dropbox

- Install Dropbox and let the initial sync finish
- Overleaf: Menu, Dropbox Sync, **Link**. Overleaf creates `Apps/Overleaf/` with one subfolder per project
- Open the project folder in VS Code and edit as normal; every save reaches Overleaf within seconds, and Overleaf edits flow back
- Compile in Overleaf as usual, or locally with `latexmk -pdf`

No commit step. Good for active coauthoring, less good if you want a history.

Back to Overleaf.

Other tools generate referee reports too

TOOL	WHAT IT IS
Refine.ink	Referee-style report on a paper PDF; paid
coarse.ink	Lightweight version of the same idea; close to free
Referee 2	Referee-report service; paid
Stanford Agentic Reviewer	Open agentic reviewer; free

A skill that lives next to your draft knows your conventions and lets you edit what it checks. The services do not.

Back to the debrief.

Bryan's eight rules: protect learning, then teach more

Protect learning

Decide what should be learned without AI, and say which assignments test it.

Give students an incentive to learn: regular, graded checks. Effort follows tests.

Do not give AI-cheatable assignments. Almost every take-home worth real credit can be done by a frontier model today.

Teach more with AI

Help students learn more efficiently: class-specific tutoring, spaced repetition, mastery learning at scale.

Use AI to improve your own teaching. Find out where you were confusing before the exam tells you.

Personalise assignments to the level each student is at. Raise standards: if drafting is cheaper, the bar moves up. Write the AI policy into the syllabus, per assignment.

Rules 1–8 of Bryan (2026). Back to the teaching recap.

Three things to build this term: a tutor, a misconception digest, levelled problem sets

Course tutor

A shared assistant (a Claude project, a custom GPT, NotebookLM) loaded with *your* lecture notes and problem sets.

Standing instructions: use the course's notation, never give the answer, ask the student to explain first, return to what they missed.

Misconception digest

Each week, export the questions students asked, anonymised.

Prompt: "Group these by underlying misconception, rank by frequency, and point to the lecture slide most likely to have caused each."

Levelled problem sets

One problem, three difficulty tiers. Assign the tier by last week's quiz result.

Prompt: "Write this problem at three levels: scaffolded, standard, extension. Same concept, full solutions, and a rubric."

Student questions are personal data. Anonymise before exporting, and check the institutional policy once. Back to the teaching recap.

A replication package is a README, a master script and portable paths

1. A README: data sources, access terms, software versions, runtime.
2. A table mapping each exhibit to the script that makes it and the file it writes.
3. One master script that runs the pipeline in order, with relative paths and seeds set.
4. The data you may ship, and access instructions for the data you may not.

Three prompts, then one check

"Read `Code/` and draft the README, including the exhibit-to-script table."

"List every input file the code reads; flag the ones not in this folder."

"Make every path relative and write `master.do`."

Then verify: *run it in a fresh folder and diff the tables against the paper. With register data that run happens on the server; the code and the README are the package.*

References

Bryan, Kevin A. 2026. *Eight Rules for Teaching in AI World*. Https://kevinbryanecon.com/teachwithai.html.